Now the issue is: we can parallelize the execution of DBSCAN on each pseudo-cluster, but then, to put together all the results, we need to check all the pairs of clusters produced by each parallel run, and all the (filtered) pairs of points in the Cartesian product between the external rings of these clusters; do we need to run these checks in a single thread on a single machine, hence being back to sequential execution?

13.2 MapReduce

For a long time, engineers have struggled to efficiently parallelize execution of algorithms like DBSCAN, at least from a practical point of view, because of both hardware limits and lack of software infrastructure. The most used distributed programming model was GRID computing, until it was realized that a different approach would make computation not just faster, but potentially more robust; that's when the *MapReduce* programming model was adopted on a large scale, after being patented and used by Google in the early 2000s.

Although there are several implementations of MapReduce, (or we should rather say: several products leveraging MapReduce to provide tools that orchestrate distributed resources to solve tasks, like Apache Hadoop, Hive or CloudDB), I believe its main value is in the model it provides, which can be applied to a plethora of tasks.

And, for this reason, we'll try to explain how it works through an example.